

**Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский университет ИТМО**



Факультет Программной Инженерии и Компьютерной Техники

**Лабораторная работа №1:
Разработка защищенного REST API с интеграцией CI/CD
по дисциплине**

«Информационная безопасность»:

**Выполнил: Исупов Никита Александрович
Группа: Р3431
Преподаватель: Рыбаков Степан Дмитриевич**

**Санкт-Петербург,
2026**

1. Используемые технологии

Приложение разработано на Java 17 с использованием Spring Boot 4.1.1. Веб-слой реализован на Spring Web MVC, механизмы доступа - на Spring Security, а работа с данными - на Spring Data JPA.

Компонент	Назначение
Spring Web MVC	REST-контроллеры и JSON API
Spring Security	JWT-аутентификация и правила доступа
Spring Data JPA	Параметризованный доступ к данным
H2 (файл / память)	Рабочая и тестовая базы данных
SpotBugs	Статический анализ Java-кода (SAST)
OWASP Dependency-Check	Поиск известных CVE в зависимостях (SCA)

2. Архитектура приложения

```
ru.nifreebie.infoseclab1
|-- configuration - конфигурация Spring Security
|-- controller - REST-контроллеры и обработка ошибок
|-- dto - объекты запросов и ответов
|-- model - JPA-сущности и перечисления
|-- repository - Spring Data JPA репозитории
|-- security - создание и проверка JWT
|-- service - бизнес-логика
`-- utils - прикладные исключения
```

3. Описание REST API

Метод	Путь	Назначение	Доступ
POST	/auth/register	Регистрация, возврат UUID пользователя	Публичный
POST	/auth/login	Проверка пароля и выдача JWT	Публичный
POST	/api/logs	Создание лога и возврат UUID	Bearer JWT
GET	/api/logs	Получение логов текущего пользователя	Bearer JWT

3.1 Пример регистрации

```
POST /auth/register
Content-Type: application/json

{
  "username": "student",
  "password": "StrongPassword123!"
}
```

При успешной регистрации сервер отвечает статусом 201 Created и возвращает UUID пользователя. Открытый пароль и его хэш в ответ не включаются.

3.2 Создание события

```
POST /api/logs
Authorization: Bearer <JWT>
Content-Type: application/json

{
  "level": "WARNING",
  "source": "authentication-service",
  "message": "Failed login attempt",
  "eventTime": "2026-09-27T12:30:00Z"
}
```

4. Реализованные меры защиты

4.1 Защита от SQL-инъекций

Доступ к данным реализован через Spring Data JPA. Параметры передаются отдельно от текста SQL-запроса, поэтому пользовательский ввод не может изменить структуру запроса.

4.2 Защита от XSS

Поля source и message проверяются по длине и обязательности, а перед возвратом экранируются методом HtmlUtils.htmlEscape. Ответы имеют тип application/json, политика CSP default-src 'none' запрещает загрузку активного содержимого.

4.3 Защита аутентификации

- пароли хранятся как BCrypt-хэши с cost factor 12;
- JWT подписывается HMAC-SHA-256, секрет поступает из JWT_SECRET;
- проверяются алгоритм, издатель, подпись и срок действия JWT;
- сервер работает без HTTP-сессии;
- ошибка входа одинакова для неверного пароля и неизвестного пользователя.

4.4 Контроль доступа и безопасные ошибки

После проверки JWT пользователь загружается из базы по UUID, а логи выбираются только по UUID владельца. Без токена защищённый ресурс возвращает 401. Для DTO используются ограничения Jakarta Bean Validation.

5. JWT-аутентификация

Защищённый запрос обрабатывается в следующей последовательности:

- клиент передаёт JWT в заголовке Authorization с префиксом Bearer;
- JwtAuthenticationFilter извлекает токен и вызывает JwtService;
- JwtService проверяет подпись, издателя и exp;
- пользователь проверяется в базе по UUID и username;
- Authentication сохраняется в SecurityContext;
- правило anyRequest().authenticated() разрешает обращение к API.

Открытыми остаются только /auth/register, /auth/login и /error. JWT содержит идентифицирующие данные и временные метки; пароль и его хэш в токен не включаются.

6. Интеграция проверок безопасности в CI/CD

GitHub Actions запускается для pull request в ветку main и для push непосредственно в main, что исключает двойной запуск pipeline для одного изменения. Pipeline выполняет сборку, SAST-проверку SpotBugs, SCA-проверку Dependency-Check и сохраняет отчёты как artifacts.

6.1 SAST: SpotBugs

SpotBugs запускается с уровнем effort Max и порогом Low. Параметр failOnError=true останавливает pipeline при обнаружении предупреждений. На рисунке 1 показан успешный запуск: BugInstance size = 0, Error size = 0, ошибок и предупреждений не обнаружено.

```
1  ▶ Run ./mvnw --batch-mode --no-transfer-progress spotbugs:check
9  [INFO] Scanning for projects...
10 [INFO]
11 [INFO] -----< ru.nifreebie:secure-log-api >-----
12 [INFO] Building SecureLog API 0.0.1-SNAPSHOT
13 [INFO] from pom.xml
14 [INFO] -----[ jar ]-----
15 [INFO]
16 [INFO] >>> spotbugs:4.9.8.1:check (default-cli) > :spotbugs @ secure-log-api >>>
17 [INFO]
18 [INFO] --- spotbugs:4.9.8.1:spotbugs (spotbugs) @ secure-log-api ---
19 [INFO] Fork Value is true
20 [INFO] Done SpotBugs Analysis....
21 [INFO] Skipping com.github.spotbugs:spotbugs-maven-plugin:4.9.8.1:spotbugs report goal
22 [INFO]
23 [INFO] <<< spotbugs:4.9.8.1:check (default-cli) < :spotbugs @ secure-log-api <<<
24 [INFO]
25 [INFO]
26 [INFO] --- spotbugs:4.9.8.1:check (default-cli) @ secure-log-api ---
27 [INFO] BugInstance size is 0
28 [INFO] Error size is 0
29 [INFO] No errors/warnings found
30 [INFO] -----
31 [INFO] BUILD SUCCESS
32 [INFO] -----
33 [INFO] Total time: 15.641 s
34 [INFO] Finished at: 2026-09-27T20:23:15Z
35 [INFO] -----
```

Рисунок 1 - Успешное выполнение SAST-проверки SpotBugs

6.2 SCA: OWASP Dependency-Check

Dependency-Check сопоставляет Maven-зависимости с базой NVD. Сборка завершается ошибкой при CVSS 7.0 или выше. Во время работы была обнаружена уязвимая версия tomcat-embed-core 11.0.24; после обновления Tomcat до 11.0.26 повторный запуск завершился успешно. Результаты представлены на рисунках 2 и 3.

Успешное выполнение SCA-проверки Dependency-Check

```
[INFO] Analysis Started
[INFO] Finished Archive Analyzer (0 seconds)
[INFO] Finished File Name Analyzer (0 seconds)
[INFO] Finished Jar Analyzer (0 seconds)
[INFO] Finished Dependency Merging Analyzer (0 seconds)
[INFO] Finished Hint Analyzer (0 seconds)
[INFO] Finished Version Filter Analyzer (0 seconds)
[INFO] Created CPE Index (1 seconds)
[INFO] Finished CPE Analyzer (3 seconds)
[INFO] Finished False Positive Analyzer (0 seconds)
[INFO] Finished NVD CVE Analyzer (0 seconds)
[INFO] Finished RetireJS Analyzer (0 seconds)
[INFO] Finished Vulnerability Suppression Analyzer (0 seconds)
[INFO] Finished Known Exploited Vulnerability Analyzer (0 seconds)
[INFO] Finished Dependency Bundling Analyzer (0 seconds)
[INFO] Finished Unused Suppression Rule Analyzer (0 seconds)
[INFO] Analysis Complete (6 seconds)
[INFO] Writing HTML report to: /home/runner/work/information-secutiry-lab1/information-secutiry-lab1/target/dependency-check-report.html
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 04:32 min
[INFO] Finished at: 2026-09-27T20:27:49Z
[INFO] -----
```

Рисунок 2 - Успешное выполнение SCA-проверки Dependency-Check

Итоговый HTML-отчёт OWASP Dependency-Check



Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

[How to read the report](#) | [Suppressing false positives](#) | [Getting Help: github issues](#)

Project: SecureLog API

ru.nifreebie:secure-log-api:0.0.1-SNAPSHOT

Scan Information ([show all](#)):

- *dependency-check version*: 12.1.8
- *Report Generated On*: Sun, 27 Sep 2026 20:44:41 GMT
- *Dependencies Scanned*: 88 (48 unique)
- *Vulnerable Dependencies*: 0
- *Vulnerabilities Found*: 0
- *Vulnerabilities Suppressed*: 0
- ...

Summary

Summary of Vulnerable Dependencies ([click to show all](#))

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
------------	-------------------	---------	------------------	-----------	------------	----------------

Dependencies (vulnerable)

This report contains data retrieved from the [National Vulnerability Database](#).

This report may contain data retrieved from the [CISA Known Exploited Vulnerability Catalog](#).

This report may contain data retrieved from the [Github Advisory Database \(via NPM Audit API\)](#).

This report may contain data retrieved from [RetireJS](#).

This report may contain data retrieved from the [Sonatype OSS Index](#).

Рисунок 3 - Итоговый HTML-отчёт OWASP Dependency-Check

7. Вывод

В результате работы разработан защищённый REST API SecureLog с регистрацией, JWT-аутентификацией, созданием логов и получением записей текущего пользователя. Идентификаторы пользователей и логов имеют тип UUID, а доступ к операциям с логами без JWT запрещён.

Применены параметризованные запросы через JPA, экранирование пользовательских строк, BCrypt, проверка подписи и срока действия JWT, stateless-конфигурация Spring Security, валидация DTO и централизованная обработка ошибок. SpotBugs и OWASP Dependency-Check встроены в GitHub Actions.

Проверки безопасности подтвердили отсутствие предупреждений SpotBugs и известных уязвимостей в итоговом наборе зависимостей. В HTML-отчёте Dependency-Check указано: проверено 88 зависимостей (48 уникальных), уязвимых зависимостей - 0, найденных уязвимостей - 0.